

Multiclass Classification and Back-propagation

Softmax, a Stable Implementation, Adam, Convolutional Layers, and Computation Graphs

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 5 · Part 2

Beyond yes or no

Part 1 gave us

- Training: `Sequential`, `compile(loss=...)`, `fit`.
- ReLU in hidden layers; output activation chosen from the range of y.
- Without a non-linear g, depth collapses.

Every output so far was one number. Today the output becomes a probability for *each* of N classes. Then we open the two black boxes inside `fit`: the optimiser, and the computation of gradients.

Today

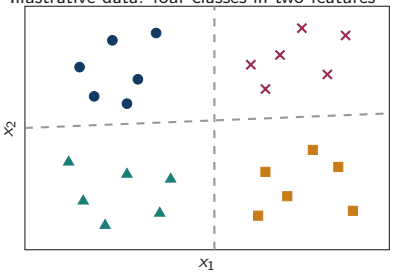
1. Multiclass classification and softmax
2. Softmax in a network: a stable implementation; multi-label output
3. Advanced optimisation: Adam
4. Additional layer types: convolution
5. Derivatives, computation graphs and back-propagation

The last section is marked optional in the syllabus but is examinable here: it is how `fit` obtains every gradient.

Multiclass Classification and Softmax

More than two classes

Illustrative data: four classes in two features



Multiclass classification

The target y takes one of $N > 2$ *discrete* values: $y \in \{1, \ldots, N\}$. Exactly one is correct for each example.

Examples

- Handwritten digit: $N = 10$.
- Diagnosis of a leaf: healthy, blast, brown spot, or blight.

Key idea

We now want an estimate of $P(y = j \mid \mathbf{x})$ for every class j — N numbers that are non-negative and sum to 1.

The softmax function

Softmax regression, N classes

One linear score per class,

$$z_j = \mathbf{w}_j \cdot \mathbf{x} + b_j, \qquad j = 1, \ldots, N,$$

turned into probabilities by

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^{N} e^{z_k}} = P(y = j \mid \mathbf{x}).$$

Exponentiation makes every score positive; dividing by the sum makes them add to 1. The largest z_j gets the largest a_j — hence "soft" max.

Worked example: $\mathbf{z} = [2,\ 1,\ 0.1]$

j	z_j	e^{z_j}	a_j
1	2	7.3891	0.6590
2	1	2.7183	0.2424
3	0.1	1.1052	0.0986
		11.2125	1.0000

$N = 2$ gives back logistic regression

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = g(z_1 - z_2)$$

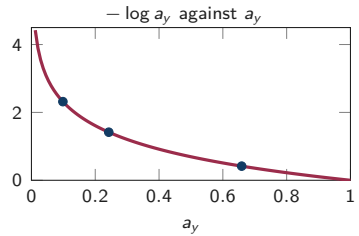
e.g. $\mathbf{z} = [2, 0.5]$: $a_1 = 0.8176 = g(1.5)$.

Cross-entropy loss

$$L(\mathbf{a}, y) = -\log a_y = \begin{cases} -\log a_1 & y = 1 \\ \vdots & \\ -\log a_N & y = N \end{cases}$$

Only the probability assigned to the *correct* class is penalised.

For $N = 2$ this is exactly binary cross-entropy: if $y = 1$ the loss is $-\log a_1$; if $y = 2$ it is $-\log a_2 = -\log(1 - a_1)$.



Same example

$L = 0.4170$ if $y = 1$; 1.4170 if $y = 2$; 2.3170 if $y = 3$.

The losses differ by exactly the gaps between the z_j: $-\log a_j = \log \sum_k e^{z_k} - z_j$.

Softmax in a Neural Network

A softmax output layer



Digits 0–9 from 28×28 images. The output layer computes

$$z_j^{[3]} = \mathbf{w}_j^{[3]} \cdot \mathbf{a}^{[2]} + b_j^{[3]}, \qquad a_j^{[3]} = \frac{e^{z_j^{[3]}}}{\sum_{k=1}^{10} e^{z_k^{[3]}}}$$

```
from tensorflow.keras.losses import \
    SparseCategoricalCrossentropy
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')])
model.compile(loss=SparseCategoricalCrossentropy())
model.fit(X, y, epochs=100)
```

Softmax is different

Sigmoid, ReLU and linear act on each z_j alone. Each softmax output a_j depends on *all* of $z_1, \ldots, z_N$, through the denominator. Raise one score and every other probability falls.

"Sparse categorical"

Categorical: y is one of N categories. *Sparse*: y is stored as a single integer (0–9), not as a length-10 one-hot vector.

Do not ship this version

It works, but it is numerically fragile. The next two slides give the version you should actually use.

Round-off: the same number, two ways

```
x1 = 2.0 / 10000
print(f"{x1:.20f}")
# 0.00020000000000000001

x2 = 1 + 1/10000 - (1 - 1/10000)
print(f"{x2:.20f}")
# 0.00019999999999977797
```

Mathematically $x_1 = x_2$. In floating point, computing a small number as the difference of two numbers near 1 loses digits.

The softmax version of the problem

For logits $\mathbf{z} = [1000, 999, 998]$, `np.exp(1000)` overflows to `inf`, and $a_1 = \texttt{inf}/\texttt{inf} = \texttt{nan}$; very negative z give $\log 0$.

Do not compute a and then $\log a$

Substitute the softmax into the loss and simplify *before* evaluating:

$$-\log a_y = \log \sum_k e^{z_k} - z_y.$$

Shift by $c = \max_k z_k$, so that every exponent is ≤ 0:

$$\log \sum_k e^{z_k} = c + \log \sum_k e^{z_k - c}.$$

Check it

$\mathbf{z} = [1000, 999, 998]$, $y = 1$:

$$L = \log(1 + e^{-1} + e^{-2}) = 0.4076$$

— finite, and computed without ever forming e^{1000}.

The recommended implementation: `from_logits=True`

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='linear')])  # z, not a
model.compile(
    loss=SparseCategoricalCrossentropy(from_logits=True))
model.fit(X, y, epochs=100)

logits = model(X_new)              # z: scores, not probs
probs  = tf.nn.softmax(logits)     # if probabilities needed
yhat   = np.argmax(logits, axis=1)
```

Binary case, same idea: `Dense(1, 'linear')` with `BinaryCrossentropy(from_logits=True)`; recover the probability with `tf.nn.sigmoid(logits)`.

Logits

The values **z** that the last layer outputs *before* the softmax (or sigmoid). `from_logits=True` tells the loss to apply the softmax itself, using the stable formula.

The model now outputs z

`model(X)` returns logits, which can be negative and do not sum to 1. Apply `tf.nn.softmax` before reporting probabilities. For the predicted class you do not need it: $\arg\max_j z_j = \arg\max_j a_j$.

Multi-label classification (several outputs)

A different question

A street camera image. Is there a **car**? A **bus**? A **pedestrian**? Any combination may be present, so the label is a vector:

$$\vec{y} = [1,\ 0,\ 1] \quad \text{(car, no bus, pedestrian)}.$$



	multiclass	multi-label
labels per example	exactly one	any number
output layer	N softmax units	N sigmoid units
outputs sum to	1	anything in $[0, N]$
loss	cross-entropy	binary cross-entropy per output, summed

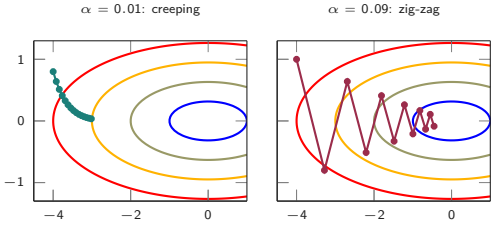
Choosing by the data, not by habit

Softmax on a multi-label task forces the classes to compete: a confident "car" pushes "pedestrian" down even when both are present.

Advanced Optimisation: Adam

One learning rate does not suit every parameter

$J = w_1^2 + 10\,w_2^2$, gradient descent from the left



x-axis w_1, y-axis w_2; 15 and 12 steps shown.

Two failure modes, one fix

Left: every step moves w_1 in the same direction, but slowly — w_1 wants a *larger* step.

Right: w_2 flips sign every step — it wants a *smaller* step, while w_1 is still happy to go faster.

Adam — adaptive moment estimation

Give every parameter its own effective step size, adjusted automatically from the history of its gradients:

- gradient keeps the same sign $\Rightarrow$ steps stay large;
- gradient oscillates $\Rightarrow$ the average cancels, steps shrink.

The Adam update, and how to use it

Adam, for one parameter w at step t

With gradient $d = \partial J / \partial w$:

$$m \leftarrow \beta_1 m + (1-\beta_1)\, d \qquad \text{mean of } d$$

$$v \leftarrow \beta_2 v + (1-\beta_2)\, d^2 \qquad \text{mean of } d^2$$

$$w \leftarrow w - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}, \qquad \hat{m} = \frac{m}{1-\beta_1^t},\ \hat{v} = \frac{v}{1-\beta_2^t}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$; $\alpha = 10^{-3}$.

$\hat{m}/\sqrt{\hat{v}}$ is near ± 1 when the gradient is consistent, so each step is about α *whatever the gradient's scale*; near 0 when it oscillates.

```
model.compile(
  optimizer=tf.keras.optimizers.Adam(
      learning_rate=1e-3),
  loss=SparseCategoricalCrossentropy(
      from_logits=True))
```

In practice

Adam is the default optimiser for neural networks. It is far less sensitive to the choice of α than plain gradient descent, but α still matters: try 10^{-2}, 10^{-3}, 10^{-4}.

Not magic

A bad loss, unscaled features or a bug in the data are not fixed by changing the optimiser. Check the learning curve first.

Additional Layer Types

Dense layers, and an alternative

Dense layer

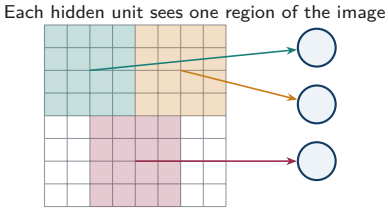
Every unit sees every activation of the previous layer. Powerful, but parameter-hungry: 784 pixels into 25 units already costs 19 625 parameters.

Convolutional layer

Each unit looks only at a **window** — a small, contiguous region of its input — and the *same* weights (a **filter**) are slid across all windows.

Why it helps.

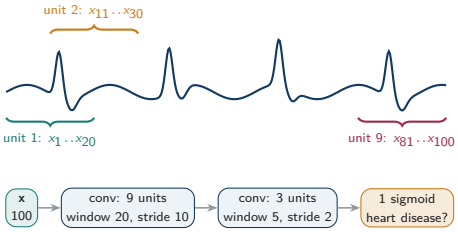
- Far fewer parameters ⇒ faster to compute.
- Needs less data, and is less prone to overfitting (Week 3).
- Builds in a useful assumption: a pattern means the same thing wherever it appears.



Hand-designing the window size, the stride (how far each window moves), and the number of filters is part of the architecture. Convolutional networks are the workhorse of computer vision.

Example: classifying a heart signal

An EKG trace: 100 samples $x_1, \ldots, x_{100}$ (schematic)



Windows

Layer 1: windows start at $1, 11, \ldots, 81$: 9 units.
Layer 2 looks at $a^{[1]}_{1..5}$, $a^{[1]}_{3..7}$, $a^{[1]}_{5..9}$: 3 units.

Parameters in layer 1

dense, $100 \to 9$	$100 \cdot 9 + 9 = 909$
own weights per window	$9 \cdot (20 + 1) = 189$
one shared filter	$20 + 1 = 21$

Key idea

A real convolutional layer uses several filters, each shared across all windows, so it detects several patterns anywhere in the signal for a few dozen parameters apiece.

Derivatives, Computation Graphs and Back-propagation

Informal definition

If nudging w by a tiny ε changes $J(w)$ by about $k\varepsilon$, then

$$\frac{dJ}{dw} = k.$$

Example: $J = w^2$ at $w = 3$, with $\varepsilon = 0.001$:

$J(3.001) = 9.006001$, a rise of $0.006001 \approx 6\varepsilon$. So $dJ/dw = 6$ — and indeed $\frac{d}{dw} w^2 = 2w = 6$.

The update $w \leftarrow w - \alpha\, dJ/dw$: a large derivative means J is sensitive to w, so take a large step; a negative one means increasing w lowers J.

Check: derivatives at $w = 2$, $\varepsilon = 0.001$

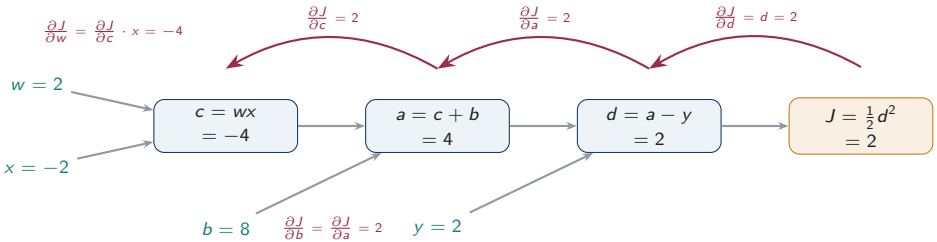
$J(w)$	$\frac{dJ}{dw}$	at $w = 2$	$\frac{J(2.001) - J(2)}{0.001}$
w^2	$2w$	4	4.001
w^3	$3w^2$	12	12.006
w	1	1	1
$1/w$	$-1/w^2$	-0.25	-0.2499

Numerical gradient check

Comparing an analytic derivative with $\big(J(w+\varepsilon) - J(w)\big)/\varepsilon$ is the standard way to test gradient code. Symbolic tools (e.g. SymPy) do the algebra if you need it.

The computation graph

Linear regression on one example: $J = \frac{1}{2}(wx + b - y)^2$, with $w = 2,\ b = 8,\ x = -2,\ y = 2$.



Forward, left to right

Compute and store every intermediate value: c, a, d, J.

Backward, right to left

Each node multiplies the derivative arriving from its right by its own local derivative (chain rule). Check: nudging w to 2.001 changes J by -0.004.

Why backward is efficient

Reuse

$\partial J/\partial a$ was computed once and then used for *both* $\partial J/\partial b$ and $\partial J/\partial c$ (and through c, for w). Every intermediate derivative is shared by everything to its left.

Cost, for N nodes and P parameters

- Nudge each parameter and re-run the forward pass: about $N \times P$ steps.
- Back-propagation: about $N + P$ steps.

With $N = 10^4$ and $P = 10^5$: 10^9 against 1.1×10^5 — the difference between impossible and instant.

Back-propagation

Compute all $\partial J/\partial$(parameter) in one backward sweep over the computation graph, applying the chain rule at each node and reusing every intermediate result.

Automatic differentiation

TensorFlow records the forward computation as a graph while it runs, then runs back-propagation over it. This is the "library does the calculus" of Part 1 — and why you only ever write the forward pass.

A larger example: two ReLU layers

Setting

One input, one unit per layer, ReLU in both, squared-error loss:

$$a^{[1]} = g(w^{[1]}x + b^{[1]}), \quad a^{[2]} = g(w^{[2]}a^{[1]} + b^{[2]})$$

$$J = \tfrac{1}{2}\left(a^{[2]} - y\right)^2$$

$x = 1$, $y = 5$; $w^{[1]} = 2$, $b^{[1]} = 0$; $w^{[2]} = 3$, $b^{[2]} = 1$.

Forward

$z^{[1]} = 2$, $a^{[1]} = 2$; $z^{[2]} = 3 \cdot 2 + 1 = 7$, $a^{[2]} = 7$; $J = \tfrac{1}{2}(7-5)^2 = 2$.

Backward (ReLU$'(z) = 1$ here, since every $z > 0$)

$\partial J/\partial a^{[2]}$	$= a^{[2]} - y$	$= 2$
$\partial J/\partial z^{[2]}$	$= 2 \cdot 1$	$= 2$
$\partial J/\partial w^{[2]}$	$= 2 \cdot a^{[1]}$	$= \mathbf{4}$
$\partial J/\partial b^{[2]}$	$= 2$	$= \mathbf{2}$
$\partial J/\partial a^{[1]}$	$= 2 \cdot w^{[2]}$	$= 6$
$\partial J/\partial z^{[1]}$	$= 6 \cdot 1$	$= 6$
$\partial J/\partial w^{[1]}$	$= 6 \cdot x$	$= \mathbf{6}$
$\partial J/\partial b^{[1]}$	$= 6$	$= \mathbf{6}$

Verify numerically

Nudging each parameter by 0.001 gives 6.0045, 6.0045, 4.0020, 2.0005 — agreeing to the size of ε.

Wrap-up and Assessment

Summary

What to remember

- Softmax turns N scores into N probabilities; with $N = 2$ it is logistic regression.
- Loss $-\log a_y$: `SparseCategoricalCrossentropy`.
- Output linear logits and pass `from_logits=True`: numerically stable.
- Multi-label: N sigmoid outputs, not softmax.
- Adam adapts a step size per parameter; it is the default optimiser.
- Convolutional units see windows and share weights.
- Back-propagation: one backward sweep, chain rule, $\approx N + P$ steps.

Formulas

$$a_j = \frac{e^{z_j}}{\sum_k e^{z_k}}, \qquad L = -\log a_y$$

$$-\log a_y = \log \sum_k e^{z_k} - z_y$$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial c} \cdot \frac{\partial c}{\partial w}$$

Next

Week 6: advice for applying machine learning — evaluating a model, train/validation/test sets, and diagnosing bias and variance.

Descriptive questions

1. Define multiclass classification and give two examples. Why can one sigmoid output not handle it? **[4]**
2. State the softmax function. Compute $\mathbf{a}$ for $\mathbf{z} = [1,\ 0,\ -1]$ and verify that the outputs sum to 1. **[6]**
3. Show that softmax with $N = 2$ is the same as logistic regression, and say what plays the role of z. **[6]**
4. Write the cross-entropy loss for softmax. For the worked example $\mathbf{z} = [2, 1, 0.1]$, which label gives the largest loss, and why? **[5]**
5. Explain why softmax is unlike sigmoid, ReLU and linear activations in how each output depends on the scores. **[4]**
6. What are logits? Explain why `from_logits=True` with a linear output layer is preferred, and how you then obtain probabilities. **[6]**

Descriptive questions (continued)

7. Distinguish multiclass from multi-label classification, giving the output layer and loss for each. **[5]**
8. Describe the Adam update. Explain how it behaves when a parameter's gradient keeps the same sign, and when it oscillates. **[6]**
9. What is a convolutional layer? A 1-D input has 200 samples; layer 1 uses windows of 40 with stride 20. How many units, and how many parameters with one shared filter? **[6]**
10. For $J = \frac{1}{2}(wx + b - y)^2$ with $w = 1$, $b = 2$, $x = 3$, $y = 1$, draw the computation graph and compute $\partial J/\partial w$ and $\partial J/\partial b$ by back-propagation. **[8]**
11. Explain why back-propagation takes about $N + P$ steps whereas finite differences take about $N \times P$. **[5]**
12. In the two-ReLU-layer example, recompute every derivative with $w^{[2]} = -1$. What happens, and why? **[8]**

Design questions

1. **Design a crop-disease classifier.** The Bangladesh Rice Research Institute wants to classify leaf photographs as healthy, blast, brown spot or bacterial blight. Specify the architecture (including any convolutional layers), every activation, the loss, the optimiser, and how you will output probabilities to farmers. **[12]**
2. **Design a news tagger.** A Bangla news portal wants to tag each article with any of 12 topics (politics, sport, economy, . . .). Design the output layer and loss, and explain what would go wrong with softmax. **[10]**
3. **Design a gradient checker.** Design a routine that verifies the gradients produced by a back-propagation implementation for an arbitrary network. Specify ε, the comparison metric, the tolerance, and how you handle the ReLU kink at 0. **[12]**
4. **Design an optimiser comparison.** Design an experiment comparing plain gradient descent and Adam on the digit data. Specify the learning rates you sweep, what you measure, and how you make the comparison fair. **[10]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§10.2** — softmax output and cross-entropy on the MNIST digits; **§10.3** — convolutional networks; **§10.7.1** — back-propagation.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§4.3.4** — multiclass logistic regression (softmax); **§5.3** — error back-propagation.

Lab (CSE 816, Week 5): softmax; multiclass; practice lab on neural networks for multiclass classification; derivatives and back-propagation.

Practice quizzes: *Multiclass Classification*; *Additional Neural Network Concepts*.